

Rapport exercice VII

I Présentation de l'exercice

Le but de l'exercice était d'implanter l'algorithme des k_moyens, et de le paralléliser à l'aide d'OpenMp.

II Implantation

J'ai d'abord commencé par définir plusieurs structures qui allaient m'être utiles :

```
typedef struct {
    Float x, y, z;
} Vec3;

typedef struct {
    Vec3 pos;
} Cluster;

typedef struct {
    Vec3 pos;
    int cluster;
} Point;

typedef struct {
    Float x, y, z;
    int count;
} Acc;
```

J'ai aussi écrit un parser, pour lire les fichiers contenant les points. Son écriture n'est pas détaillée ici, pour essayer de rester concis.

Ensuite, j'ai écrit une implantation non parallélisée des k_moyens. Je ne détaille pas son implantation, mais elle peut être retrouvée sur le [GitHub](#) du projet.

Pour l'implantation parallélisée, j'ai décidé d'utiliser une réduction, cela n'a pas été facile à implémenter, j'ai d'abord essayé avec une réduction personnalisée grâce à `omp declare reduction(...)`, mais le débogage était trop compliqué, et le code trop dur à lire, j'ai donc opté pour une réduction à base d'un bloc single.

Voici les détails de l'implantation : Premièrement, on alloue autant de `Cluster` que demandé par l'utilisateur de la fonction, et on alloue un tableau d'accumulateurs `Acc`, qui contiendra les accumulateurs locaux à chaque thread. La variable `changed` va nous permettre de savoir quand l'algorithme aura convergé vers sa solution finale.

<kmeans> ≡

```
int kmeans(Point *points, int points_counts, int clusters_count, int nb_t){
    Cluster *clusters = malloc(clusters_count * sizeof(Cluster));
    Acc *local_acc_array = calloc(nb_t * clusters_count, sizeof(Acc));

    int changed = 1;
    iter = 0; //> Utile à des fins de débogages
    //...
```

Ensuite nous entrons dans la partie parallélisée du programme. Chaque thread récupère un pointeur vers son tableau d'accumulateurs local. Puis l'algorithme itère jusqu'à converger.

<kmeans> + ≡

```
#pragma omp parallel num_threads(nb_t)
{
    // Getting the local cluster
    int tid = omp_get_thread_num();

    Acc *acc = &local_acc_array[tid * clusters_count];

    while (changed) {
```

```

    <itération kmeans>
}

free(local_acc_array);
free(clusters);
return 0;
}

```

Chaque itération consiste à réinitialiser les accumulateurs, à trouver le cluster le plus proche, et à réduire. Une barrière est nécessaire entre les deux dernières étapes, elles dépendent toutes deux l'une de l'autre.

<itération kmeans> $\equiv$

```

    <réinitialisation des accumulateurs>
    <cluster le plus proche>
    <réduction>

```

La réinitialisation est relativement simple, chaque membre est mis à 0.

<réinitialisation des accumulateurs> $\equiv$

```

for (int k = 0; k < clusters_count; k++) {
    acc[k].x = acc[k].y = acc[k].z = 0;
    acc[k].count = 0;
}

```

Il s'agit ici d'une simple recherche d'argument minimum par rapport aux clusters partagés, et d'une accumulation dans les accumulateurs locaux. On notera l'utilisation du carré de la distance euclidienne avec dx dy dz et $dist = dx * dx + dy * dy + dz * dz$. Ici nous prenons le carré afin d'économiser un calcul de racines carrée, en effet $\|u\| > \|v\| \Leftrightarrow \|u\|^2 > \|v\|^2$, avec u et v représentant les $[dx \ dy \ dz]$ calculées préalablement.

Aucun contrôle sur la concurrence des données n'est nécessaire pour l'accumulation, grâce à la localité des accumulateurs.

<cluster le plus proche> $\equiv$

```

#pragma omp for
for (int i = 0; i < points_counts; i++) {
    Point p = points[i];
    // Finding the nearest cluster
    int nearest = 0;
    Float min_dist = Float_INFTY;

    for (int k = 0; k < clusters_count; k++) {
        Float dx = p.pos.x - clusters[k].pos.x;
        Float dy = p.pos.y - clusters[k].pos.y;
        Float dz = p.pos.z - clusters[k].pos.z;

        Float dist = dx * dx + dy * dy + dz * dz;

        if (dist < min_dist) {
            min_dist = dist;
            nearest = k;
        }
    }
    // Save current result
    points[i].cluster = nearest;
}

```

```
//Accumulation
acc[nearest].x += p.pos.x;
acc[nearest].y += p.pos.y;
acc[nearest].z += p.pos.z;
acc[nearest].count++;
}
// Implicit Barrier
```

Et enfin, nous procédons à l'accumulation. Le code peut sembler long par rapport à la partie parallélisée, mais il ne s'agit en fait que d'une boucle de complexité $O(K \times T)$ avec K le nombre de clusters, et T le nombre de threads, la complexité sera donc toujours inférieure à celle de la boucle parallélisée précédemment (dans tous les cas, le nombre de clusters doit être inférieur au nombre de points, et le nombre de threads ne peut être trop élevé pour des raisons matérielles).

< réduction >≡

```
#pragma omp single
{
    changed = 0;

    for (int k = 0; k < clusters_count; k++) {
        Float sx = 0, sy = 0, sz = 0;
        int sc = 0;

        for (int t = 0; t < nb_t; t++) {
            Acc *a = &local_acc_array[t * clusters_count + k];
            sx += a->x;
            sy += a->y;
            sz += a->z;
            sc += a->count;
        }

        if (sc > 0) {
            Vec3 new_center = {
                sx / sc,
                sy / sc,
                sz / sc,
            };

            if (Float_abs(new_center.x - clusters[k].pos.x) > EPSILON ||
                Float_abs(new_center.y - clusters[k].pos.y) > EPSILON ||
                Float_abs(new_center.z - clusters[k].pos.z) > EPSILON)
                changed = 1;

            clusters[k].pos = new_center;
        }
    }
    iter++;
}
// Implicit Barrier
```

III Calcul des temps

Afin de calculer les temps d'exécution, je me suis aidé d'un script Python, et de la sortie standard du programme en c.

Toutes les données sont donc automatiquement compilées dans un fichier Json.

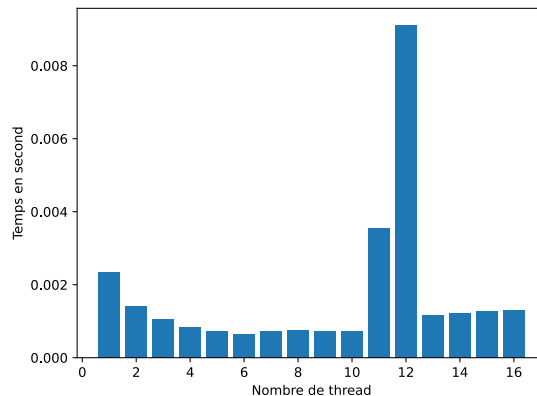
Pour une bonne cohérence au niveau des mesures, j'ai fait une moyenne sur 10 exécutions pour chaque test.

III.1 Résultats

À chaque fois, j'ai estimé un s de la loi d'Amdahl, grâce à la formule trouvée sur la page Wikipedia [de la loi d'Amdahl](#).

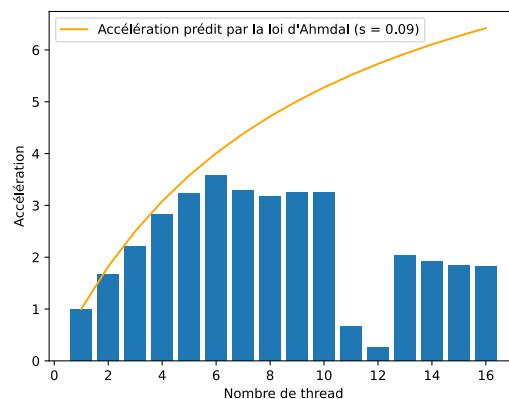
III.1.a Temps pour 10000 points

Voici un graphe représentant les temps enregistrés pour un jeu de données de 10 000 points :



On peut voir une amélioration du temps au fur et à mesure, malgré un petit jeu de données. On observe tout de même un pic à 12 threads, plusieurs raisons se présentent, cela peut être la cohérence du cache mise à mal par l'écriture dans le buffer commun d'accumulateurs locaux, ou aussi le fait que la machine (Chapitre IV.1) utilisée pour les mesures possède 12 threads logiques.

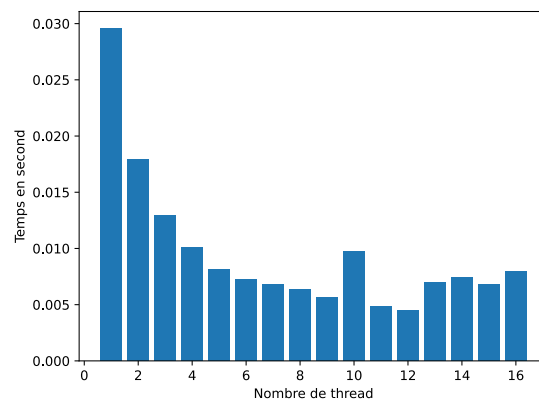
Regardons maintenant l'accélération :



L'accélération correspond à ce qui est observé sur le graphe, et on voit bien que sur les premiers nombres de threads, cela suit la loi d'Amdahl théorique.

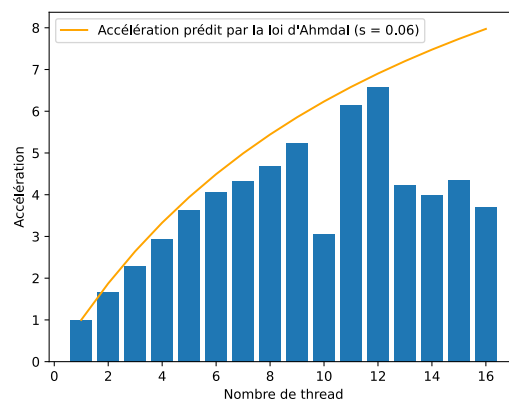
III.1.b Temps pour 100000 points

Maintenant les temps pour un jeu à 100 000 points :



Ici le jeu de données étant plus conséquent, l'amélioration est d'autant plus remarquable. On remarque un pic à 10 threads, probablement pour les mêmes raisons que le jeu de données précédent.

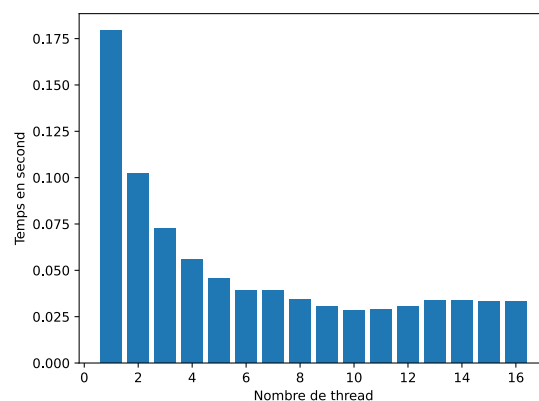
Regardons maintenant l'accélération :



L'accélération correspond à ce qui est observé sur le graphe, et on voit bien que sur les premiers nombres de threads, cela suit la loi d'Amdahl théorique.

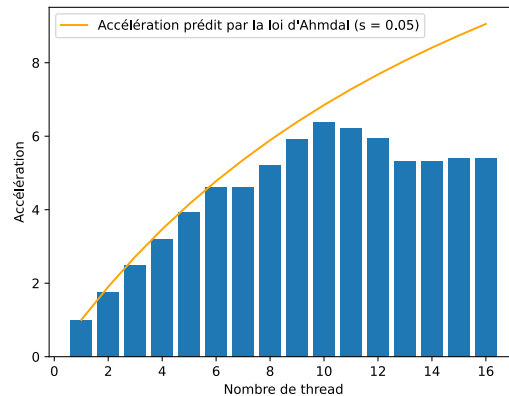
III.1.c Temps pour 1 000 000 points

Et enfin les temps pour un jeu à 1 000 000 points :



Il s'agissait du plus gros jeu de données fourni, l'évolution des temps est dans la continuité de ce qui était observé précédemment. On remarque que les pics à 10 et 12 threads ne se voient plus, cela doit être dû à la taille du jeu de données, qui écrase ces perturbations.

Et quant à l'accélération :



La correspondance avec la loi d'Amdahl est encore plus frappante ici, bien que l'on remarque une limite atteinte au-delà de 12 thread, encore une fois, sûrement due à la machine.

IV Conclusion

L'accélération des k moyens nous permet d'obtenir de bien meilleures performances, et se fait avec un ratio d'instructions non séquentielles (le s de la loi d'Amdahl) assez faible.

Donc pour un passage à l'échelle sur une machine à 64 cœurs, par exemple, en considérant le s observé dans les graphes, nous aurions une accélération théorique calculée par : $\mathcal{S}_{64} = \frac{1}{0.05 - \frac{1-0.05}{64}} \approx 28.44$.

IV.1 Machine utilisée

Tous les temps ont été enregistrés sur un ordinateur portable avec un processeur x86-64 de 6 cœurs physiques dont deux virtuels par cœur, cadencé à ~4 GHz.

IV.2 Données récoltées

Toutes les mesures effectuées sont stockées dans un document *json* récupérer grâce à la sortie standard du programme, et à un script Python, trouvable sur le [GitHub](#).

IV.3 Code

L'entièreté du code se trouve sur le [GitHub](#) dans le fichier Ex7.

IV.4 Notice sur l'utilisation de l'IA générative

L'IA générative n'a pas été utilisée pour l'écriture du rapport, elle a été utilisée à des fins de débogage, le code reste écrit et non générer.